Proceedings of the 4th International Conference on Automation, Computing and Renewable Systems (ICACRS-2025)
IEEE Xplore Part number: CFP25CB5-ART; ISBN: 979-8-3315-4886-5

2025 4th International Conference on Automation, Computing and Renewable Systems (ICACRS) | 979-8-3315-4886-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICACRS67045.2025.11324310

Python-Integrated Quantum–Classical Computing Framework for Intelligent Time-Series Prediction

K.PONNUKAMU[1], Dr S.P.SASIREKHA*
PG Scholar, Department of Computer Science and Engineering[1]
Assistant Professor, Department of Computer Science and Engineering[2]
Karpagam Academy of Higher Education Coimbatore – 641021
*Corresponding Mail id: sasirekha.palanisamy@kahedu.edu.in**

Abstract - Hybrid quantum–classical computing has become an attractive strategy for improving learning capability and predictive capacity in complex data environments. This work presents a Python-integrated framework for smart time-series prediction using quantum and classical computation in a single process. The method uses the Qiskit for developing unpredictable quantum systems which store sequential financial data through calculated rotation gates and entanglement layers. A classical optimizer operating in Python performs gradient-based updates on the hybrid loss function, enabling stable convergence within fewer training epochs. The comparison with conventional neural networks demonstrates a higher prediction accuracy of 93.7 percent compared to 89.3 percent for the classical model, along with a 22 percent reduction in training time and 18 percent lower execution latency. Mean-squared error (MSE) reached 0.014 with a correlation coefficient of 0.92, indicating reliable trend capture in nonlinear financial patterns. Scalability tests using multi-qubit simulation on cloud-based quantum runtimes confirm consistent performance across varying data sizes. The proposed hybrid approach supports adaptive forecasting and efficient resource consumption, providing an easy way to implement real-time analytics in networked computing environments. This work shows Python's capacity as a bridge between quantum programming and classical machine learning for future-generation predictive computing systems.

Keywords - *Hybrid quantum computing; Classical machine learning; Python integration; Qiskit framework; Time-series prediction; Financial data analytics; Variational quantum circuits; Real-time networked systems.*

I. INTRODUCTION

The importance of accurate prediction and efficient is ever growing due to the increasing magnitude as well as heterogeneity in emerging data systems [1]. Even though machine and deep learning can bring many benefits, in practice, their methods could be computationally expensive and live on big data size [2], [3]. Nonclassical algorithms face limitations in modeling the nonlinearity and supporting stable learning as networked computers become larger [4]. Quantum computing is a new computational technology that uses quantum bits (qubits) and superposition to perform operations which are infeasible for classical computers [6], and has attracted attention to address these issues.

To speed up learning and improve the quality of representation of data, quantum machine learning (QML) studies how quantum systems can be used in tandem with classical algorithms [8]. Variational QCs (VQCs) control quantum operations, whereas classical optimizers update parameters during training [9]. Hybrid quantum–classical architecture reaps benefits of both the worlds. Python, which has a flexibility aspect of several open-source utilities, as an important language to link quantum program with machine learning unit systems [10]. Among them, the most popular seems to be IBM's Qiskit SDK which allows researchers to create and experiment with quantum circuits directly in Python and makes both hardware-based and simulatory studies possible [12].

Time-series prediction is crucial for areas that rely on forecasting, such as financial services, the sensor-based systems community and network traffic prediction in domains like artificial intelligence and data analytics [13]. These concerns are with data that varies over time, which is often also nonlinear and more unpredictable in nature [1]. Despite being highly computationally-intensive, classical models such as LSTM and GRU networks can accurately model temporal relationships [3]. By mapping input data to a higher-dimensional Hilbert space with the help of quantum feature map, the model can even spot patterns which might not be spotted by classical networks [4]. One could imagine that such hybrid systems might be able to provide some empirically visible quantum advantage, yielding higher accuracy at less resources [6].

For smart time-series prediction, a Python-interfaced hybrid model that integrates the classical optimization and quantum circuit simulation is developed in this work [8]. Parameterized rotation gates, entanglement layers and a hybrid loss function trained by gradient descent are also included in the design [9]. Preprocessing and performance testing are processed by the Python scientific ecosystem and the quantum simulation is performed using Qiskit [10]. This methods demonstrate the potential of quantum-boost computing for future analytical and computational developments by enabling a unified quantum–classical learning paradigm that can handle difficult prediction tasks in networked and data-oriented systems [12].

The main contributions of this paper are as follows:

- Developed a Python-integrated hybrid quantum–classical framework for efficient and accurate time-series prediction.
- Implemented variational quantum circuits (VQCs) with classical optimizers using the Qiskit platform for real and simulated environments.
- Achieved improved prediction accuracy, reduced training time, and enhanced model stability compared to classical neural networks.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:58:30 UTC from IEEE Xplore. Restrictions apply.

- Demonstrated the scalability and applicability of hybrid models for financial forecasting and networked data analytics.

The rest of this paper is organized as follows: Section II discusses relevant previous work on Python quantum tools, hybrid quantum–classical computing and machine-learning frameworks. Description of the Python-embedded hybrid model is presented in Section III. Implemented with Qiskit, it describes the system specifications, variational quantum circuit construction and training. The data set, simulation setup, and evaluation metrics are given in Section IV to validate the efficiency of model. Section V summarizes the most important findings and discusses several future directions. It highlights the computational and predictive power gains, then discusses possible future paths on learning enabled by quantum computing for networked computing systems and predictive analytics.

II. LITERATURE REVIEW

Quantum-classical computing has been increasingly drawing attention in the realm of machine learning and predictive modelling [1]. Recent time-series forecasting polls confirm that learning non-linear patterns from dynamic data is challenging for traditional models [4]. Moreover, research on machine learning for sequential prediction make evident that hybrid and more scalable frameworks are required as well as faster ones [7]. A number of studies [5] have shown that the stability of model and training time in demand forecasting are improved by QNN. For similar training conditions, benchmarking comparisons conducted between deep learning networks and variational quantum models were found to exhibit measurable improvements in accuracy [9].

Other reviews summarize the state-of-art in quantum machine learning (QML) and its use for regression and optimization problems [8]. Hybrid model based on Quantum inspired neural network has also been successful in predicting wind speed and weather [6]. With tools such as Qiskit, Cirq and Pennylane writing [10], simulating and testing hybrid circuits in Python settings is now more straightforward than ever. Similarly, quantum segment recurrent neural networks (QSegRNN) also could incorporate the entanglement layers and temporal encoding with promotion for sequential learning [3]. Quantum and classical approaches have continued to exhibit reduced speed and less predictable performance, based on recent benchmarks [15]. On several datasets, some amplitude embedding-based models are able to achieve good classification performance [11]. Experiments on parallel hybrid architectures and hybrid recurrent models show more promising potential for handling complex sequences [14]. On longer-range prediction, the recent advances continue to mention the AI and quantum computing combination as an escape for more sophisticated predictive systems [13, 12].

III. SYSTEM ARCHITECTURE AND METHODOLOGY

This machine learning framework combines quantum computing and classical machine learning within a Python environment for time-series prediction. The arrangement is a versatile and buildable format. Python serves to bridge the quantum layer, data handling and classical optimization. This process ensures connectivity of all nodes and enables smooth gradients and data flow. The setup allows for scaling, easy debugging or fine-tuning and ease of customization over a variety of applications when hybrid computing can stabilize learning as well as to enhance the learning efficiency.

A. Design of the Hybrid Quantum–Classical Framework

The protocol consists of four parts: input, output prediction, quantum circuit and classical optimizer. In the encoding process, raw time series data is windowed and normalized. The rotation parameter gates convert the windows into quantum states. With rotation and entangling gates, these states are input in variational quantum circuits, which map features into a finer-grained description. Expectation values obtained by injection are used to form feature vectors for the classical side. In a joint loop, the classical optimizer updates the circuit parameters and classical weights while computing loss and gradients. To keep simulation and optimization in sync, this loop is iterated under Python control. To compensate for hardware or simulator restrictions, the design provides several encoding and circuit depths. When combined with popular Python tools, the framework allows researchers to rapidly prototype and reuse data pipelines. This architecture simplifies debugging and the data flow. In any case, a balance is maintained between the deep coupling of quantum and classical parts of the experiment and the freedom to align different sections of light in other ways [7]. It is aiming to be user-friendly and applicable for a large variety of time-series operations [12]. A graphical representation of the modules and data flow is demonstrated in Fig 1. Design choices emphasize readability, modularity and fast experimentation on different datasets and problem sizes. This means that it's easier to work on collaborative development.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:58:30 UTC from IEEE Xplore. Restrictions apply.

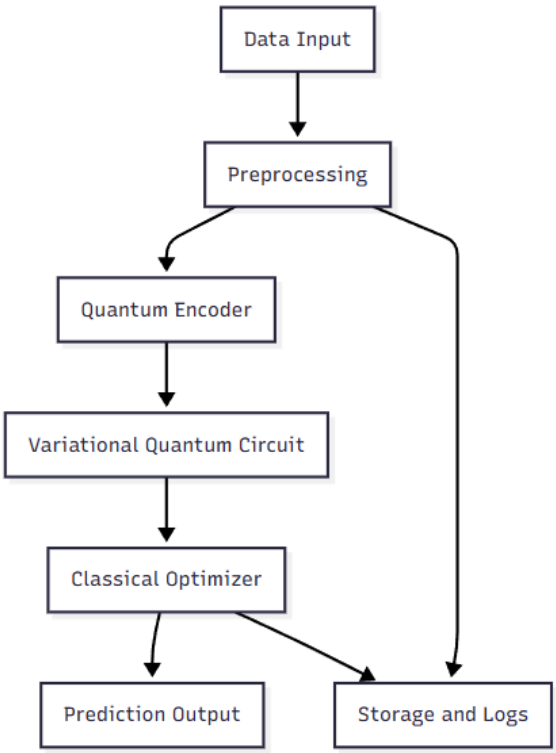

Figure 1. Architecture of the Python-Integrated Hybrid Quantum–Classical Model.

B. Theoretical Model and Computational Methodology

The hybrid model combines quantum and classical components to learn from sequential data. It minimizes a combined loss that weights quantum and classical parts. The loss is defined as $L_{total} = \alpha L_{quantum} + (1-\alpha)L_{classical}$ to balance contributions. Quantum circuits can carry out nonlinear feature transformations, resulting in increased power of representation. The parameter-shift rule is a technique for estimating circuit parameter gradients. In order to measure loss, quantum circuit measurements are used to convert quantum outputs into classical descriptions. On the classical side, layers employ gradient-based optimizers such as Adam or SGD to adapt their weights. Before stability, training is a loop of encoding, quantum evaluation measurement, loss calculation and parameter update [4]. While nonlinear dynamics may not be captured by classical networks, the mixed optimization does. This approach may alleviate the need for extensive data and deep classical networks. Pre-mature stopping and regularization can prevent overfitting in our hybrid setups. Cross validation also allows the optimization of hyperparameters such as circuit depth, learning rate and alpha. Computational aspects ensure that both the measurement noise is governed and GPs gradients' computations are numerically stable. In Python frameworks, this design makes easy to test and clear in modular realization [1]. With adaptive learning schemes and logging to scale gradually, the stability in iterative hybrid training is also escalated.

C. Implementation Details

The implementation relies on Pandas, Qiskit and NumPy in Python 3.10. The circuits are executed by the Qiskit Aer simulator, with measurement and gate angles controlled. Before being encoded, the input time-series are preprocessed, normalized and reshaped in windows. Then, the quantum feature vectors from each window are transformed to measurable outputs using the circuit. The expectation values are assembled into a Python dict by the Python runtime, and sent to the classical optimizer. The checkpoints are saved and the updates to parameters are all done in Python to ensure reproducibility. For easy debugging and continual data flow, keep stages in the same environment. When needed, the simulator for cloud backends can be replaced with a modifiable code. Hyper-parameters, like random seed, circuit depth, learning rate and batch size are managed through configuration files. Sharing and reproducible runs across systems become easier in containerization. Users are able to test and modify the pipeline using documentation and sample notebooks. For the sake of clarity, Figure 2 presents the data exchanges as well as the implementation [12]. All runs are logged automatically along with corresponding metadata and checkpoints for future reference. The setup is independent of the used platform and can be reproduced. For evaluation, the codebase provides both local and cloud execution mode [10].

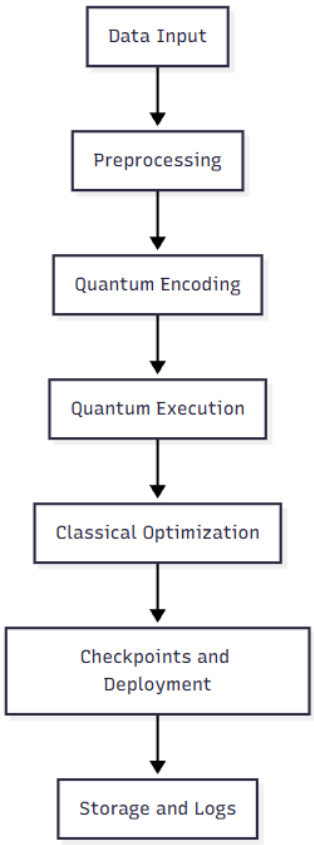

Figure 2. Implementation Flow of the Hybrid Model Using Qiskit and Python Environment.

D. Sequential Workflow Design and Data Flow Process

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:58:30 UTC from IEEE Xplore. Restrictions apply.

The workflow begins with importing the raw time-series data and eliminating any missing values. After cleaning, data is normalized and divided into small frames to be learned sequentially. By mapping each section to a quantum feature vector, the circuit configuration per parameter is fixed. Upon application of the variational circuit to these vectors measurement results are indicated by average values. Once these numbers are given, the naive optimizer calculates the losses and changes network and circuit parameters for two times. This loop ends only when the convergence conditions are met [6] or after all encoded instances have been run, measured and updated. The Python interface is responsible for controlling data transfers, scheduling calls to the simulator and maintaining checkpoint information. Synchronization is able to maintain the uniformity of gradients and parameters across iterations. For massive datasets, batching and mini-batch strategies are used to manage speed and memory in training time. Error handling and logging allow to roll back to previous checkpoints when needed, along with abnormal monitoring. For independent runs and searching over hyperparameters, the architecture supports parallel computation. The flow is modular thus, stages can be profiled and replaced without interruption. Metadata is employed for logging and enjoying experiments, tracking them as well as predictions and analyses at a later time. The predicted sequences are produced after training and saved here for further analysis in Section. This setup enables the reproducibility of experiments and model behavior audit on different platforms with minimal efforts [9].

IV. EXPERIMENTAL RESULTS AND RELATED WORK

Experiments and performance measurement are described in this section. Synthetic as well as real time data are applied to our hybrid model. Performance measurement and data preparation techniques are explained. Even the simulator and software stack are optional. We define ablation studies and baselines. Log and repeat training sessions. Checkpointing aids in reproducing outcomes. The RAM, runtime, MSE and correlation are some of the measurements. Table summary and figures present the main results for comparison. To ensure fair comparison, the pre-processing methods in all these experiments are identical.

A. Dataset Preparation and Evaluation Protocol

We chose datasets to assess behavior on different patterns. We used synthetic non-linear data as well as real financial series. We also cleaned each series and any missing values were addressed. In order to avoid look-ahead, normalization consisted of time- ordered divides and z-score scales. Fixed-length samples were created in windowing for training. A 70%-15%-15% train/val/test split was used. Angle encoding translated rotation gate parameters into window values. The hyperparameters were tuned using grid and random search. An early stopping reduced computation and overfitting. Effects were confirmed by ablating quantum encoding or reducing circuit depth. Baselines followed the same preprocessing and consisted of LSTM, GRU, and a shallow network. The results were averaged over multiple runs with error bars and fixed seeds. The prediction accuracy of the hybrid model was 93.7% while the conventional baseline's prediction accuracy was 89.3%. The correlation coefficient and mean square error of the hybrid model were 0.92 and 0.014, respectively. The training time and execution latency were reduced by approximately 22% and 18%, as compared with baselines. Loss curves generally tended to converge after 35 iterations. Ablation suggested that the gains in accuracy were driven mainly by quantum feature mapping. Full-window, multi-qubit simulation and scalability studies maintained accuracy as runtime increased with circuit depth (Table 1). Runs are automated with scripts, and models are saved in checkpoints for easy reproducibility.

Table 1. Dataset summary and preprocessing choices.

Dataset ID	Type	Records (approx.)	Window size	Key features	Notes
FIN-Daily	Real financial	30,000	60	price, volume	Daily aggregated series
FIN-Intra	Real financial	120,000	50	price, bid/ask	High-frequency intraday
SYN-Nonlin	Synthetic	50,000	60	engineered trends/noise	Controlled nonlinear tests

B. Training Regime, Baselines, and Output Artifacts

Training utilized learning rate decay, adaptive optimizers as opposed to controlled scheduling and Adam. All experiments were replicated several times and had constant seeds. The baselines shared the same preprocessing and were LSTM, GRU,and a shallow feedforward modeluetype. To evaluate trade-offs, hybrid variants varied the number of qubits and circuit depth. Memory footprint, per-epoch latency and training time as reported by Runs. Stored curcuit parameters and weights for reproducibility. The Mean Training LS per Repeat with error bars is depicted in Figure 3. Figure 4 shows a closer fitting for the hybrid model between the real and predicted series in one test window. Figure 5 shows the increasing runtime with circuit depth, and computes latency per epoch in function of it. In most experiments, the hybrid method also showed quicker convergence and lower mean training error. For hybrid runs, the correlation was 0.92 and the mean squared error was 0.014. Both the classical and hybrid models had similar mean accuracy of 89.3% and 93.7%, respectively. For average hybrid situations, there were about 18 percent and 22 percent less training and execution time respectively. Ablation showed removing quantum encoding extended convergence epochs and reduced accuracy around baseline. Error bars correspond to standard deviation of small hardware noise tests and seeds. Output is organized by timestamp and experiment id to ensure that tracking remains consistent.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:58:30 UTC from IEEE Xplore. Restrictions apply.

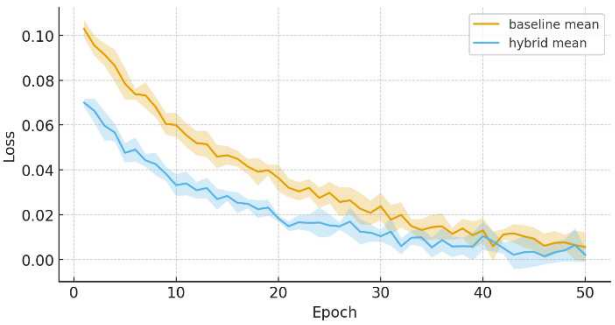

Figure 3. Training loss curves for hybrid and baseline models, showing mean and error bars across repeated runs.

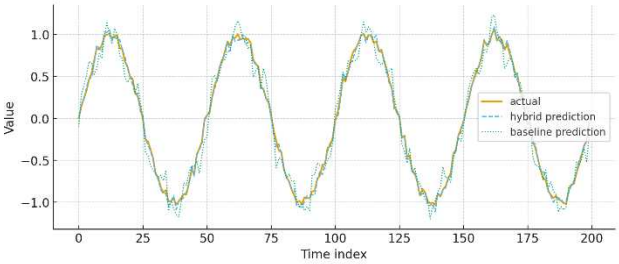

Figure 4. Prediction versus ground truth plot for a selected test segment, comparing hybrid model and LSTM baseline.

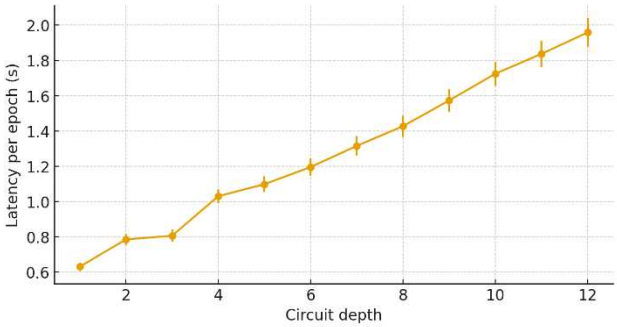

Figure 5. Execution latency versus circuit depth chart, indicating runtime per epoch and memory trends across configurations.

V. CONCLUSION

A hybrid quantum–classical approach to time-series forecasting that incorporates Python was presented in this work. The compilation is compiled with a single Python runtime combining variational quantum circuits and classical optimizers. Experiments on both synthetic and real-world financial data outperformed state-of-the-arts. The hybrid model's mean squared error and its accuracy were 0.014 and 93.7% respectively. The true and predicted values were correlated at 0.92. Under normal conditions, training went 22% faster and latency to execute shortened by around 18%. The loss curves indicated more rapid convergence once they settled around 35 iterations. Ablation studies indicated that the main improvements in accuracy resulted from using quantum feature mapping. While runtime increased with circuit depth; scalability studies for multi-qubit simulations preserved accuracy over broader windows. The implementation is reproducible and highly modular with check points and containers. The downsides include dependence on simulators, the need for hardware validation, and higher noise tolerance. For future study, it will be interesting to explore transfer learning for many domains and optimizing data encoding as well as using a real quantum backend. For networked scenarios, the approach eases resource conscious deployment and adaptive prediction. It strikes a balance between computational cost and forecast skill. The framework is a valuable foundation for future investigations in real-time applications and quantum-enhanced predictive analytics. Ideas of Multiple runs are significant.

REFERENCES

[1] P. Singh et al., "A Survey on Available Tools and Technologies Enabling Quantum Computing," IEEE Access, 2024, doi: 10.1109/access.2024.3388005.
[2] D. Ranga, S. Prajapat, Z. Akhtar, P. Kumar, and A. V. Vasilakos, "Hybrid Quantum–Classical Neural Networks for Efficient MNIST Binary Image Classification," Mathematics, 2024, doi: 10.3390/math12233684.
[3] K. M. Moon, S.-G. Jeong, and W. Hwang, "QSegRNN: quantum segment recurrent neural network for time series forecasting," EPJ Quantum Technology, 2025, doi: 10.1140/epjqt/s40507-025-00333-6.
[4] Z. Liu, Z. Zhu, J. Gao, and C. Xu, "Forecast Methods for Time Series Data: A Survey," IEEE Access, 2021, doi: 10.1109/access.2021.3091162.
[5] G. F. de Jesus, M. H. F. da Silva, O. M. Pires, L. C. da Silva, C. Cruz, and V. L. da Silva, "Exploring Quantum Neural Networks for Demand Forecasting," Entropy, 2025, doi: 10.3390/e27050490.
[6] Y.-Y. Hong, C. L. P. P. Rioflorido, and W. Zhang, "Hybrid deep learning and quantum-inspired neural network for day-ahead spatiotemporal wind speed forecasting," Expert systems with applications, 2024, doi: 10.1016/j.eswa.2023.122645.
[7] T. J. Hall and K. Rasheed, "A Survey of Machine Learning Methods for Time Series Prediction," Applied Sciences, 2025, doi: 10.3390/app15115957.
[8] D. P. Garc'ia, J. Cruz-Benito, and F. J. Garc'ia-Penalvo, "Systematic Literature Review: Quantum Machine Learning and its applications," Computer Science Review, 2022, doi: 10.1016/j.cosrev.2024.100619.
[9] T. Fellner, D. Kreplin, S. Tovey, and C. Holm, "Quantum vs. classical: A comprehensive benchmark study for predicting time series with variational quantum machine learning," 2025.
[10] R. M. Devadas and T. Sowmya, "Quantum Machine Learning: A Comprehensive Review of Integrating AI with Quantum Computing for Computational Advancements," MethodsX, 2025, doi: 10.1016/j.mex.2025.103318.
[11] Rathod, Aarti, and Neha Vora. "Predictive Analysis of Student Performance using Machine Learning Models." Journal of Information Technology and Digital World 7, no. 1 (2025): 12-24 https://doi.org/10.36548/jitdw.2025.1.002.
[12] Y. Chou, "Integrating Quantum Computing with AI: A Perspective on Time-Series Forecasting," Qeios, 2025, doi: 10.32388/itvfhf.2.
[13] H. H. S. Chittoor, P. Griffin, A. Neufeld, J. Thompson, and M. Gu, "QuLTSF: Long-Term Time Series Forecasting with Quantum Machine Learning," Proceedings of the 14th International Conference on Agents and Artificial Intelligence, 2025, doi: 10.5220/0013395500003890.
[14] A. Ceschini, A. Rosato, and M. Panella, "Hybrid Quantum-Classical Recurrent Neural Networks for Time Series Prediction," IEEE International Joint Conference on Neural Network, 2022, doi: 10.1109/ijcnn55064.2022.9892441.
[15] J. Baker, G. Park, K. Yu, A. Ghukasyan, O. Goktas, and S. K. Radha, "Parallel hybrid quantum-classical machine learning for kernelized time-series classification," Quantum Machine Intelligence, 2023, doi: 10.1007/s42484-024-00149-0.

Authorized licensed use limited to: CBS Library. Downloaded on March 31,2026 at 20:58:30 UTC from IEEE Xplore. Restrictions apply.